React – JSON-server and Firebase Real Time Database

THEORY EXERCISE

Question 1: What do you mean by RESTful web services?

RESTful Web Services are web services that follow the principles of REST (Representational State Transfer) to allow communication between a client and a server using HTTP requests.

They are commonly used in web applications to send and receive data in JSON format.

Key Features

- Uses standard HTTP methods
    - GET – Retrieve data
    - POST – Create new data
    - PUT/PATCH – Update existing data
    - DELETE – Remove data
- Stateless communication (each request is independent)
- Uses URLs to access resources
- Usually returns data in JSON format

2. What is JSON Server? How is it used in React?

JSON Server is a simple tool that allows developers to create a fake REST API using a JSON file. It is commonly used for testing frontend applications without a real backend.

Features

- Creates a full REST API automatically
- Uses a simple db.json file
- Supports GET, POST, PUT, DELETE

- Useful for React development and testing

3. How do you fetch data from a JSON Server API in React?

In React, we fetch data using **fetch()** or **axios** inside the **useEffect() hook**.

Role of fetch()

- Built-in JavaScript function
- Sends **HTTP requests**
- Receives **API responses**
- Converts response to JSON

Role of Axios

- External library for **API requests**
- Easier syntax than fetch
- Automatic **JSON parsing**
- Better **error handling**

4. What is Firebase? What features does Firebase offer?

Firebase is a Backend-as-a-Service (BaaS) platform developed by Google that helps developers build web and mobile applications without managing their own servers.

It provides ready-made backend services such as database, authentication, and hosting.

Key Features of Firebase

1. Realtime Database
   Synchronizes data across users in real time.
2. Cloud Firestore
   Scalable NoSQL cloud database.

3. Authentication
   Provides login using email, Google, Facebook, etc.
4. Firebase Hosting
   Host web apps quickly and securely.
5. Cloud Functions
   Run backend code without managing servers.
6. Cloud Storage
   Store images, videos, and files.
7. Analytics
   Track user activity and app performance.

5. Importance of Handling Errors and Loading States in React APIs

When fetching data from APIs, the application should manage **loading and error states** to improve user experience.

1. Loading State

While data is being fetched, users should see a **loading indicator**.

Benefits:

- Improves user experience
- Shows that the application is working

2. Error Handling

Errors may occur due to:

- Network issues
- API server errors
- Wrong API endpoints

LAB EXERCISE •

Task 1:

• Create a React component that fetches data from a public API (e.g., a list of users) and displays it in a table format.

• Create a React app with Json-server and use Get , Post , Put , Delete & patch method on Json-server API.

• Task 2:

• Create a React app crud and Authentication with firebase API.

• Implement google Authentication with firebase API.

• Task 3:

• Implement error handling and loading states for the API call. Display a loading spinner while the data is being fetched.

Task 1

1. Create a React Component that fetches data from a Public API and displays it in a Table

```
npx create-react-app api-table

cd api-table

npm start

import React, { useEffect, useState } from "react";


function UsersTable() {

  const [users, setUsers] = useState([]);


  useEffect(() => {

    fetch("https://jsonplaceholder.typicode.com/users")

      .then((response) => response.json())

      .then((data) => setUsers(data));
```

```
}, []);

return (
  <div>
    <h2>User List</h2>

    <table border="1" width="80%">
      <thead>
        <tr>
          <th>ID</th>
          <th>Name</th>
          <th>Email</th>
          <th>City</th>
        </tr>
      </thead>

      <tbody>
        {users.map((user) => (
          <tr key={user.id}>
            <td>{user.id}</td>
            <td>{user.name}</td>
            <td>{user.email}</td>
            <td>{user.address.city}</td>
          </tr>
        ))}
```

```
      </tbody>

    </table>
   </div>
  );
}

export default UsersTable;
import UsersTable from "./UsersTable";

function App() {
  return (
    <div>
      <UsersTable />
    </div>
  );
}

export default App;
```

Task 2

React CRUD + Authentication with Firebase

Step 1: Install Firebase

Step 2: Create Firebase Project

Go to
https://console.firebase.google.com

1 Create project
2 Enable Authentication
3 Enable Google Sign-in

Firebase is developed by Google.

```
import { initializeApp } from "firebase/app";

const firebaseConfig = {
  apiKey: "API_KEY",
  authDomain: "PROJECT_ID.firebaseapp.com",
  projectId: "PROJECT_ID",
};

const app = initializeApp(firebaseConfig);

export default app;
import { getAuth, signInWithPopup, GoogleAuthProvider } from
"firebase/auth";
import app from "./firebase";

const auth = getAuth(app);
const provider = new GoogleAuthProvider();

function Login() {

  const googleLogin = () => {
```

```
    signInWithPopup(auth, provider)
      .then((result) => {
        console.log(result.user);
      });
  };

  return (
    <div>
      <button onClick={googleLogin}>Login with Google</button>
    </div>
  );
}

export default Login;
```

Task 3

Error Handling and Loading State

```
import React, { useState, useEffect } from "react";

function DataFetch() {

  const [data, setData] = useState([]);
  const [loading, setLoading] = useState(true);
  const [error, setError] = useState("");

  useEffect(() => {
```

```
  fetch("https://jsonplaceholder.typicode.com/posts")

    .then((res) => res.json())

    .then((data) => {
      setData(data);
      setLoading(false);
    })

    .catch((err) => {
      setError("Error fetching data");
      setLoading(false);
    });

}, []);

if (loading) {
  return <h2>Loading...</h2>;
}

if (error) {
  return <h2>{error}</h2>;
}
```

```
  return (
    <div>
      <h2>Posts</h2>

      {data.slice(0,5).map((post) => (
        <p key={post.id}>{post.title}</p>
      ))}

    </div>
  );
}

export default DataFetch;
```

Context API

Question 1: What is the Context API in React? How is it used to manage global state across multiple components?

The Context API in React is a built-in feature that allows developers to share data (state) across multiple components without passing props manually at every level.

Normally in React, data is passed from parent to child using props. When many nested components need the same data, this process becomes complex and is called prop drilling. The Context API solves this problem by creating a global state that can be accessed by any component in the application.

How Context API Manages Global State

1. Create a Context object using createContext().
2. Wrap components with a Context Provider.
3. Store the shared data inside the provider.
4. Any component can access this data using useContext().

Advantages

- Avoids prop drilling
- Makes state sharing easier
- Improves code readability
- Useful for global data such as user authentication, themes, language settings
- Ex: import { createContext } from "react";
- const UserContext = createContext();

Question 2: Explain how createContext() and useContext() are used in React for sharing state.

In React, **createContext()** and **useContext()** are used together to share data between components.

1. createContext()

createContext() is used to **create a new context object**.
It acts as a container that stores the shared state.

```
import { createContext } from "react";

export const ThemeContext = createContext();
```

2. useContext()

useContext() is a React hook used to **access the value stored in a context** from any component.

```
import { useContext } from "react";

import { ThemeContext } from "./ThemeContext";


function ComponentB() {

  const theme = useContext(ThemeContext);


  return <h1>Theme: {theme}</h1>;

}
```

State Management (Redux, Redux-Toolkit or Recoil)

THEORY EXERCISE

Question 1: What is Redux, and why is it used in React applications? Explain the core concepts of actions, reducers, and the store

Redux is a **state management library** used with React applications to manage the **global state of an application in a predictable way**.

In large applications, many components need to share and update the same data. Redux helps manage this shared data from a **centralized location called the store**, making the application easier to maintain and debug.

Why Redux is Used in React

- Manages **global state efficiently**
- Avoids **prop drilling**
- Makes state changes **predictable**
- Helps in **debugging and testing**
- Maintains a **single source of truth**

- **Core Concepts of Redux**
- **1. Actions**
- An **action** is a JavaScript object that describes **what happened in the application**. It contains a **type property** and sometimes additional data.
- **2. Reducers**
- A **reducer** is a function that determines **how the state should change based on the action received**.
- It takes the **current state and action as input** and returns a **new updated state**.

3. Store

The **store** is the central place where the **entire application state is stored**.

It:

- Holds the current state
- Allows state updates using **dispatch()**
- Provides access to state using **getState()**

Question 2: How does Recoil simplify state management in React compared to Redux?

Recoil is a modern state management library for React developed by Meta (Facebook). It simplifies state management by providing a **more flexible and easier way to manage global state compared to Redux**

How Recoil Simplifies State Management

1. Less Boilerplate Code

Redux requires many files such as:

- Actions
- Reducers
- Store configuration

Recoil reduces this complexity and allows developers to manage state with **simple hooks**.

How Recoil Simplifies State Management

1. Less Boilerplate Code

Redux requires many files such as:

- Actions
- Reducers
- Store configuration

Recoil reduces this complexity and allows developers to manage state with **simple hooks**.

3. Direct Hook-Based Access

In Recoil, components can directly access and update state using hooks.

LAB EXERCISE

Task 1

Counter Application using Redux

```
npm install redux react-redux

const initialState = {

  count: 0

};
```

```
function counterReducer(state = initialState, action) {
  switch (action.type) {

    case "INCREMENT":
      return { count: state.count + 1 };

    case "DECREMENT":
      return { count: state.count - 1 };

    default:
      return state;
  }
}

export default counterReducer;
import React from "react";
import { useSelector, useDispatch } from "react-redux";

function Counter() {

  const count = useSelector((state) => state.count);
  const dispatch = useDispatch();

  return (
```

```
    <div>

      <h2>Counter: {count}</h2>

      <button onClick={() => dispatch({ type: "INCREMENT" })}>
        Increment
      </button>

      <button onClick={() => dispatch({ type: "DECREMENT" })}>
        Decrement
      </button>

    </div>
  );
}

export default Counter;
import React from "react";
import Counter from "./Counter";
import store from "./store";
import { Provider } from "react-redux";

function App() {
  return (
    <Provider store={store}>
```

```
      <Counter />
    </Provider>
  );
}

export default App;
```

Task 2

npm install recoil

```
import { atom } from "recoil";

export const todoState = atom({
  key: "todoState",
  default: []
});
import React, { useState } from "react";
import { useRecoilState } from "recoil";
import { todoState } from "./todoState";

function Todo() {

  const [todos, setTodos] = useRecoilState(todoState);
  const [task, setTask] = useState("");

  const addTask = () => {
```

```
  setTodos([...todos, { text: task, completed: false }]);
  setTask("");
};

const toggleTask = (index) => {
  const newTodos = [...todos];
  newTodos[index].completed = !newTodos[index].completed;
  setTodos(newTodos);
};

const deleteTask = (index) => {
  const newTodos = todos.filter((_, i) => i !== index);
  setTodos(newTodos);
};

return (
  <div>

    <h2>Todo List</h2>

    <input
      value={task}
      onChange={(e) => setTask(e.target.value)}
    />
```

```
      <button onClick={addTask}>Add</button>

      <ul>
        {todos.map((todo, index) => (
          <li key={index}>
            <span
              style={{
                textDecoration: todo.completed ? "line-through" : "none"
              }}
              onClick={() => toggleTask(index)}
            >
              {todo.text}
            </span>

            <button onClick={() => deleteTask(index)}>
              Delete
            </button>

          </li>
        ))}
      </ul>

    </div>
  );
}
```

```
export default Todo;
import React from "react";
import { RecoilRoot } from "recoil";
import Todo from "./Todo";

function App() {
  return (
    <RecoilRoot>
      <Todo />
    </RecoilRoot>
  );
}

export default App;
```

Task 3

CRUD Application using Redux Toolkit

```
npm install @reduxjs/toolkit react-redux
import { createSlice } from "@reduxjs/toolkit";

const userSlice = createSlice({

  name: "users",
```

```
  initialState: [],

  reducers: {

    addUser: (state, action) => {
      state.push(action.payload);
    },

    deleteUser: (state, action) => {
      return state.filter(user => user.id !== action.payload);
    },

    updateUser: (state, action) => {
      const index = state.findIndex(
        user => user.id === action.payload.id
      );
      state[index] = action.payload;
    }

  }

});

export const { addUser, deleteUser, updateUser } = userSlice.actions;
```

```
export default userSlice.reducer;
import { configureStore } from "@reduxjs/toolkit";
import userReducer from "./userSlice";

const store = configureStore({
  reducer: {
    users: userReducer
  }
});

export default store;
import React, { useState } from "react";
import { useDispatch, useSelector } from "react-redux";
import { addUser, deleteUser } from "./userSlice";

function Users() {

  const [name, setName] = useState("");
  const users = useSelector((state) => state.users);
  const dispatch = useDispatch();

  const handleAdd = () => {
    dispatch(addUser({ id: Date.now(), name }));
    setName("");
  };
```

```
return (
  <div>

    <h2>User CRUD</h2>

    <input
      value={name}
      onChange={(e) => setName(e.target.value)}
    />

    <button onClick={handleAdd}>Add</button>

    <ul>
      {users.map(user => (
        <li key={user.id}>
          {user.name}

          <button
            onClick={() => dispatch(deleteUser(user.id))}
          >
            Delete
          </button>

        </li>
```

```
        ))}
      </ul>

    </div>
  );
}

export default Users;
```